BRNO UNIVERSITY OF TECHNOLOGY FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

Robotics and AI

MPC-MAP Assignment No. 6 - Report

Author: Martin Kříž

Date: 25. 4. 2026

Task 1

I selected the dynamic programming algorithm from the lecture, specifically value iteration, of which the pseudocode was shown on the lecture slide. The result of the first attempt is on Figure 2, the path doesn't keep any distance from the walls.

So I added the stochastic part to the algorithm, the result is on Figure 3. The stochastic formula works but still the value is influenced more by the cost function than the stochastic model, even though the model is forward - 30%, left and right – 35%. The stochastic characteristic is visible only before the turn, because the values of the cost function are very similar, so the stochastic model takes a part.

Task 2

So I implemented a function that gives extra values to the pixels that are close to the wall. One pixel is 0.2 m, so two pixels should be enough, but the principle of this method is based on values, it depends on how high the penalty for the pixel near the wall will be (too low – robot goes short but through thin spaces, too high – algorithm may not converge). The visualization of pixel penalty is on Figure 1. The final results are on Figure 4, now there is a significant improvement in keeping distance from the walls. Then I added diagonal types of actions to get a more optimal path, the results are on Figure 5.

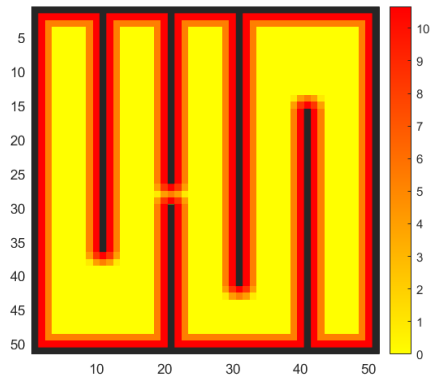

Fig. 1 – Penalty values for pixels near the wall

Task 3

I chose gradient descent path smoothing from the lecture. I'm breaking the iteration process when the change is under the bound. I set parameter α a little higher than β for a smoother path.

When the *plan_path* function is called, if it's the first time, *value_iteration* is called to get the map policy, then I don't need to call this function anymore. My *get_path* function just follows the policy signs that I have for the whole map, and I can easily replan the path (e.g. if the robot makes some stochastic motion). Anyway there is actually no need to get the path all the way to the goal, it would be enough to get it to the point that the pure_pursuit algorithm is following, but I did it for better visualization.

Appendix

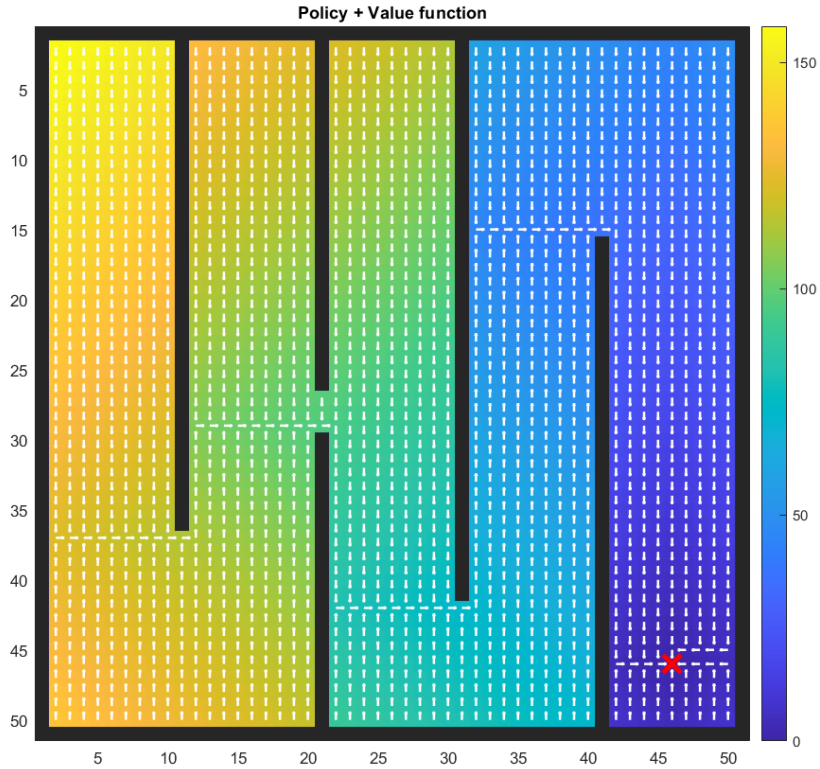

Fig. 2 – Policy and value function, without stochastic model and wall penalization

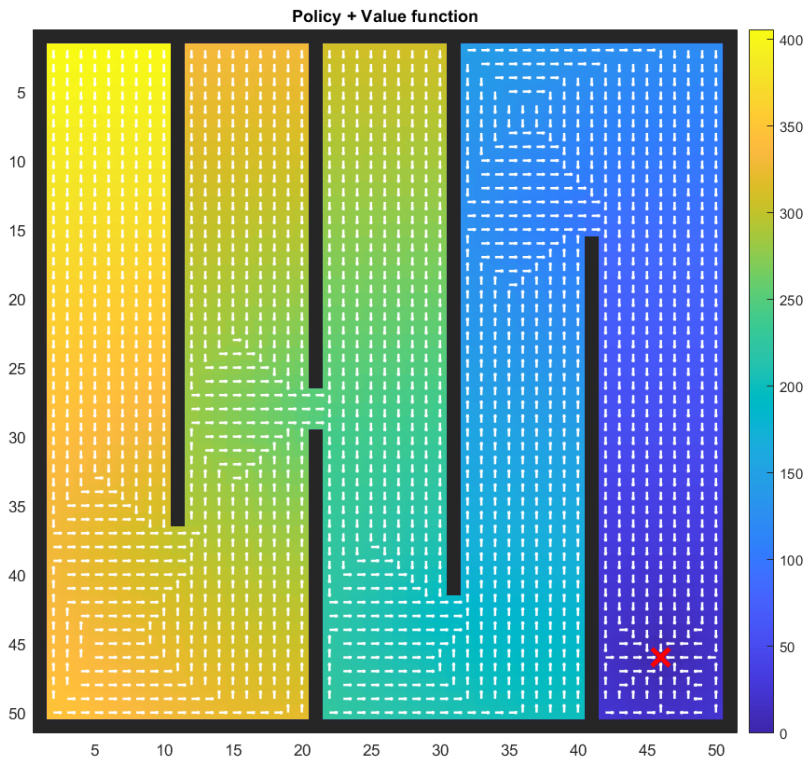

Fig. 3 – Policy and value function, with stochastic model and without wall penalization

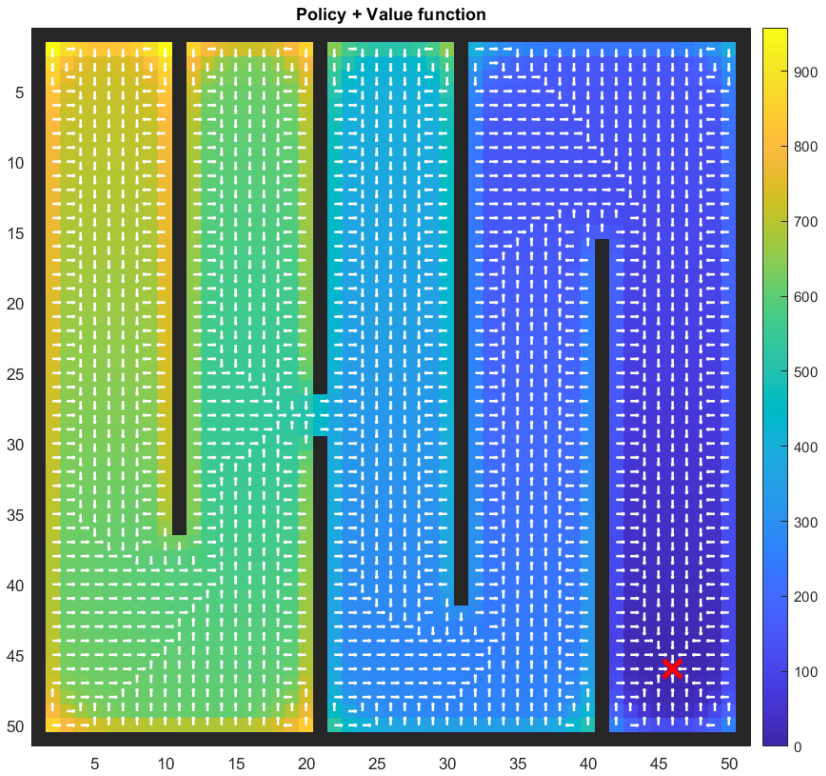

Fig. 4 – Policy and value function, with stochastic model and wall penalization

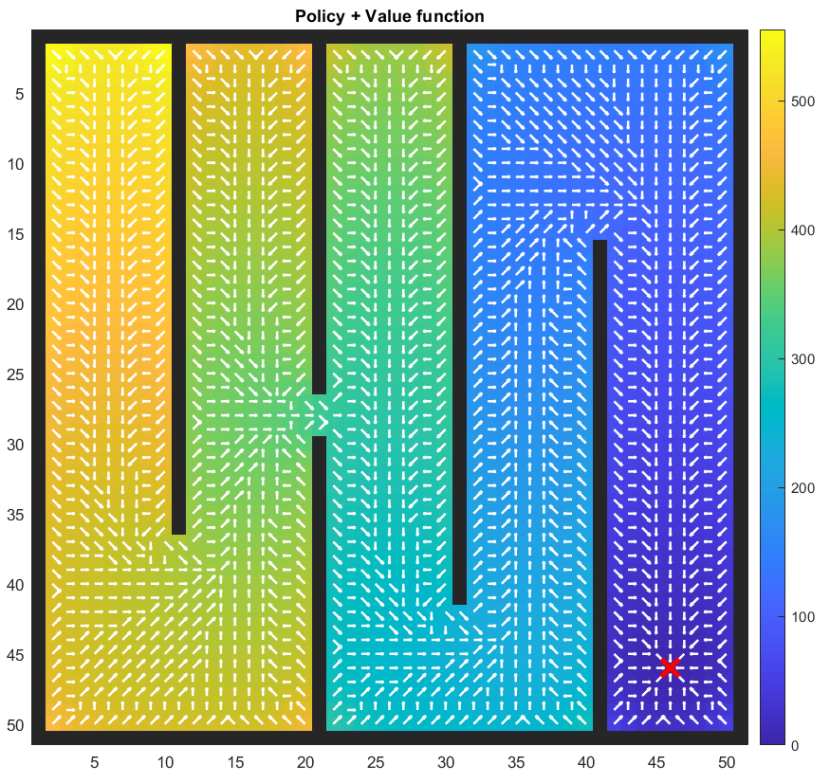

Fig. 5 – Policy and value function, same as on Fig. 4 but with diagonal actions